

```
import pandas as pd
```

```
df=pd.read_csv('/content/Salary_dataset.csv')  
df
```

Unnamed: 0	YearsExperience	Salary
0	0	1.2 39344.0
1	1	1.4 46206.0
2	2	1.6 37732.0
3	3	2.1 43526.0
4	4	2.3 39892.0
5	5	3.0 56643.0
6	6	3.1 60151.0
7	7	3.3 54446.0
8	8	3.3 64446.0
9	9	3.8 57190.0
10	10	4.0 63219.0
11	11	4.1 55795.0
12	12	4.1 56958.0
13	13	4.2 57082.0
14	14	4.6 61112.0
15	15	5.0 67939.0
16	16	5.2 66030.0
17	17	5.4 83089.0
18	18	6.0 81364.0
19	19	6.1 93941.0
20	20	6.9 91739.0
21	21	7.2 98274.0
22	22	8.0 101303.0
23	23	8.3 113813.0
24	24	8.8 109432.0
25	25	9.1 105583.0
26	26	9.6 116970.0
27	27	9.7 112636.0
28	28	10.4 122392.0
29	29	10.6 121873.0

```
df.head()
```

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

```
df.tail()
```

	Unnamed: 0	YearsExperience	Salary
25	25	9.1	105583.0
26	26	9.6	116970.0
27	27	9.7	112636.0
28	28	10.4	122392.0
29	29	10.6	121873.0

```
x=df.YearsExperience
y=df.Salary
```

```
x=df['YearsExperience'].values
y=df['Salary'].values
```

```
x
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
y
```

```
array([ 39344.,  46206.,  37732.,  43526.,  39892.,  56643.,  60151.,
        54446.,  64446.,  57190.,  63219.,  55795.,  56958.,  57082.,
        61112.,  67939.,  66030.,  83089.,  81364.,  93941.,  91739.,
        98274., 101303., 113813., 109432., 105583., 116970., 112636.,
        122392., 121873.])
```

one Method To Converting Into Array

```
x=df.iloc[:, 1].values
```

```
x
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
Y=df.iloc[:, -1].values
```

Y

```
array([ 39344., 46206., 37732., 43526., 39892., 56643., 60151.,
        54446., 64446., 57190., 63219., 55795., 56958., 57082.,
        61112., 67939., 66030., 83089., 81364., 93941., 91739.,
        98274., 101303., 113813., 109432., 105583., 116970., 112636.,
        122392., 121873.])
```

```
import numpy as np
```

```
X = np.array(x)
Y = np.array(y)
```

Y

```
array([ 39344., 46206., 37732., 43526., 39892., 56643., 60151.,
        54446., 64446., 57190., 63219., 55795., 56958., 57082.,
        61112., 67939., 66030., 83089., 81364., 93941., 91739.,
        98274., 101303., 113813., 109432., 105583., 116970., 112636.,
        122392., 121873.])
```

```
X = X.reshape(-1, 1)
```

X

```
array([[ 1.2],
       [ 1.4],
       [ 1.6],
       [ 2.1],
       [ 2.3],
       [ 3. ],
       [ 3.1],
       [ 3.3],
       [ 3.3],
       [ 3.8],
       [ 4. ],
       [ 4.1],
       [ 4.1],
       [ 4.2],
       [ 4.6],
       [ 5. ],
       [ 5.2],
       [ 5.4],
       [ 6. ],
       [ 6.1],
       [ 6.9],
       [ 7.2],
       [ 8. ],
       [ 8.3],
       [ 8.8],
       [ 9.1],
       [ 9.6],
       [ 9.7],
       [10.4],
       [10.6]])
```

```
Y=Y.reshape(-1,1)
Y
```

```
array([[ 39344.],
       [ 46206.],
       [ 37732.],
       [ 43526.],
       [ 39892.],
       [ 56643.],
       [ 60151.],
       [ 54446.],
       [ 64446.],
       [ 57190.],
       [ 63219.],
       [ 55795.],
       [ 56958.],
       [ 57082.],
       [ 61112.],
       [ 67939.],
       [ 66030.],
       [ 83089.],
       [ 81364.],
       [ 93941.],
       [ 91739.],
       [ 98274.],
       [101303.],
       [113813.],
       [109432.],
       [105583.],
       [116970.],
       [112636.],
       [122392.],
       [121873.]])
```

```
X_mean = X.mean()
X_std = X.std()

X_standardized = (X - X_mean) / X_std
```

```
X_standardized
```

```
array([[-1.51005294],
       [-1.43837321],
       [-1.36669348],
       [-1.18749416],
       [-1.11581443],
       [-0.86493538],
       [-0.82909552],
       [-0.75741579],
       [-0.75741579],
       [-0.57821647],
       [-0.50653674],
       [-0.47069688],
       [-0.47069688],
       [-0.43485702],
       [-0.29149756],
       [-0.1481381 ],
       [-0.07645838],
       [-0.00477865],
```

```
[ 0.21026054],  
[ 0.2461004 ],  
[ 0.53281931],  
[ 0.6403389 ],  
[ 0.92705781],  
[ 1.03457741],  
[ 1.21377673],  
[ 1.32129632],  
[ 1.50049564],  
[ 1.5363355 ],  
[ 1.78721455],  
[ 1.85889428]])
```

```
X_min = X.min()  
X_max = X.max()  
  
X_norm = (X - X_min) / (X_max - X_min)
```

X_norm

```
array([[0.          ],  
       [0.0212766 ],  
       [0.04255319],  
       [0.09574468],  
       [0.11702128],  
       [0.19148936],  
       [0.20212766],  
       [0.22340426],  
       [0.22340426],  
       [0.27659574],  
       [0.29787234],  
       [0.30851064],  
       [0.30851064],  
       [0.31914894],  
       [0.36170213],  
       [0.40425532],  
       [0.42553191],  
       [0.44680851],  
       [0.5106383 ],  
       [0.5212766 ],  
       [0.60638298],  
       [0.63829787],  
       [0.72340426],  
       [0.75531915],  
       [0.80851064],  
       [0.84042553],  
       [0.89361702],  
       [0.90425532],  
       [0.9787234 ],  
       [1.          ]])
```

```
m = np.random.randn()  
c = np.random.randn()  
  
print("Randomly initialized slope:", m)  
print("Randomly initialized intercept:", c)
```

```
Randomly initialized slope: -1.1591996518698213  
Randomly initialized intercept: 1.8504858946224827
```

```
def predict(X, m, c):  
    y_pred = m * X + c  
    return y_pred
```

```
y_pred = predict(X_norm, m, c)
```

```
# Display first 5 predictions  
print("First 5 predicted values:")  
print(y_pred[:5])
```

```
First 5 predicted values:  
[[0.9824248 ]  
 [0.98682503]  
 [0.99122525]  
 [1.00222581]  
 [1.00662604]]
```

```
def compute_cost(y, y_pred):  
    n = len(y)  
    cost = (1 / (2 * n)) * np.sum((y_pred - y) ** 2)  
    return cost
```

```
cost = compute_cost(Y, y_pred)
```

```
print("Initial Cost (MSE):", cost)
```

```
Initial Cost (MSE): 3251470307.197192
```

```
y_pred = predict(X_norm, m, c)
```

```
# Compute cost for given m and c  
cost = compute_cost(y, y_pred)  
  
print("Cost for given parameters:")  
print("Slope (m):", m)  
print("Intercept (c):", c)  
print("Cost (MSE):", cost)
```

```
Cost for given parameters:  
Slope (m): 8.593797504050817e-06  
Intercept (c): 76003.99999584729  
Cost (MSE): 10897488926.0
```

```
m=0.0  
c=0.0
```

```
y_pred=predict(X,m,c)
```

```
cost=compute_cost(Y,y_pred)  
print("Cost for m=",m,"and c=",c,"is:",cost)
```

[Show hidden output](#)

```
learning_rate=0.01
epochs=1000
print("Learning Rate:", learning_rate)
print("Epochs:", epochs)
```

```
Learning Rate: 0.01
Epochs: 1000
```

```
n=len(y)
```

```
cost_history = []
```

```
# Gradient Descent loop
for epoch in range(epochs):

    # Step 1: Prediction
    y_pred = predict(X_norm, m, c)

    # Step 2: Compute gradients
    dm = (1 / n) * np.sum((y_pred - y) * X_norm)
    dc = (1 / n) * np.sum(y_pred - y)

    # Step 3: Update parameters
    m = m - learning_rate * dm
    c = c - learning_rate * dc

    # Step 4: Compute and store cost
    cost = compute_cost(y, y_pred)
    cost_history.append(cost)

    # Print cost occasionally
    if epoch % 100 == 0:
        print(f"Epoch {epoch}: Cost = {cost}")
```

[Show hidden output](#)

```
# Store final trained parameters

final_slope = m
final_intercept = c

print("Final Model Parameters:")
print("Slope (m):", final_slope)
print("Intercept (c):", final_intercept)
```

```
Final Model Parameters:
Slope (m): 1.1158593813333068e-10
Intercept (c): 76003.99999999993
```

```
def predict_salary(years_experience, m, c, X_norm):
```

```
    # Predict salary
    salary_pred = m * X_norm + c
    return salary_pred
```



```
# Example: Predict salary for 5 years of experience
years = 5

# Define final_slope and final_intercept from the already existing m and c
final_slope = m
final_intercept = c

predicted_salary = predict_salary(
    years,
    final_slope,
    final_intercept,
    X_norm
)

print(f"Predicted salary for {years} years of experience:")
print(predicted_salary[0][0])
```

```
Predicted salary for 5 years of experience:
76003.99999584729
```

```
# Predict outputs using trained model
y_pred_final = predict(X_norm, final_slope, final_intercept)

# Calculate Mean Squared Error
mse = np.mean((y - y_pred_final) ** 2)

print("Mean Squared Error (MSE):", mse)
```

```
Mean Squared Error (MSE): 726499261.7333333
```

```
import matplotlib.pyplot as plt
```

```
# Generate predictions using trained model
y_pred_final = predict(X_norm, final_slope, final_intercept)

# Plot original data points
plt.scatter(X, y, label="Actual Data")

# Plot regression line
plt.plot(X, y_pred_final, label="Regression Line")

# Labels and title
plt.xlabel("Years of Experience")
```